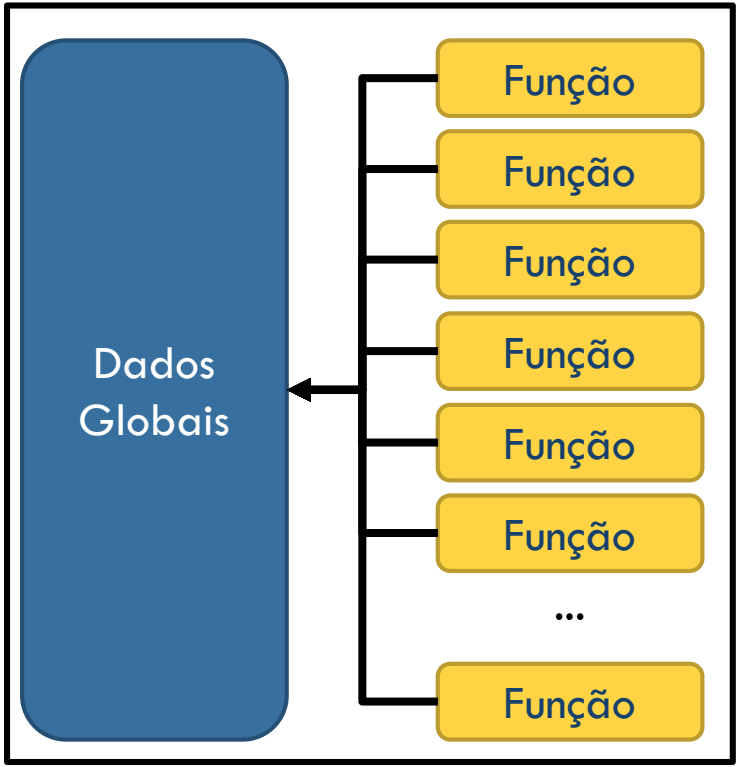
ECA304 PROGRAMAÇÃO ORIENTADA A OBJETOS

Prof. Anderson H. Moreira

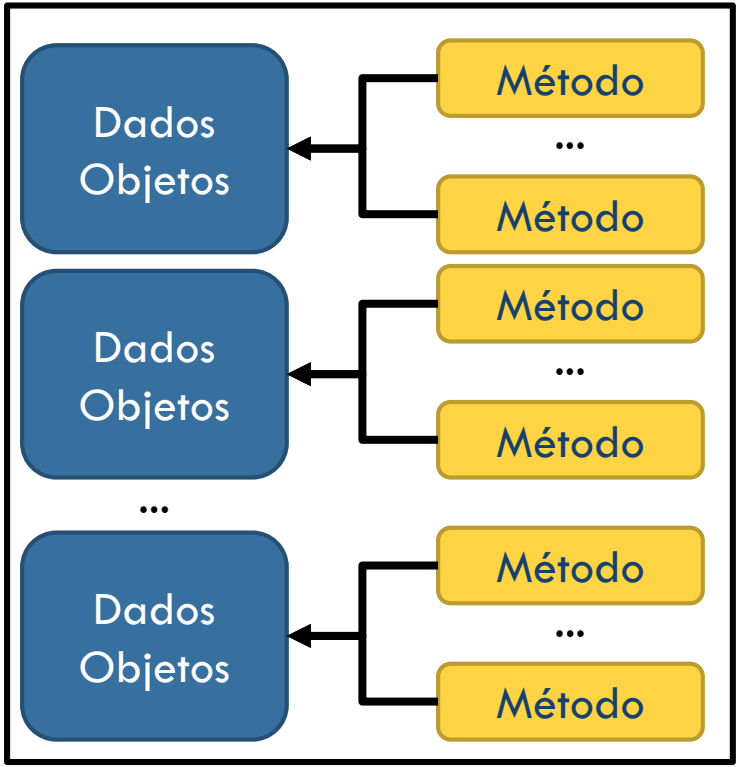
Aula 06: Classes e Objetos I

Estrutural vs Orientação a Objetos

Programação Estrutural



Programação Orientada a Objetos



Programação Orientação a Objetos

- Vantagens:
  - Tenta representar o mundo real
  - Reaproveitamento de código
  - Melhor para aplicações de grande porte
  - Manutenção e interpretação facilitada
- Desvantagens:
  - Execução mais lenta

Programação Orientação a Objetos – Termos

Polimorfismo
Classes
Herança
Sub-classes
Atributos
ProgramaçãoOrientadaAObjetos
Instância
Métodos
SobrecargaDeMétodos

Um Exemplo Fácil de Entender

Pokédex data

National №	009
Type	WATER
Species	Shellfish Pokémon
Height	5'3" (1.60m)
Weight	188.5 lbs (85.5 kg)
Abilities	Torrent Rain Dish (hidden ability)
Local №	009 (Red,Blue,Yellow,FireRed,LeafGreen) 234 (Gold,Silver,Crystal) 239 (HeartGold,SoulSilver) 088 (X,Y)
Japanese	Kamekkusu

Training

EV yield	3 Special Defense
Catch rate	45 (5.9% with PokéBall, full HP)
Base Happiness	70 (normal)
Base EXP	210
Growth Rate	Medium Slow

Breeding

Egg Groups	Monster, Water 1
Gender	87.5% male, 12.5% female
Egg cycles	21 (minimum 5,355 steps)

Base stats

			Min	Max
HP	79		268	362
Attack	83		153	291
Defense	100		184	328
Sp. Atk	85		157	295
Sp. Def	105		193	339
Speed	78		144	280
Total	530		Min	Max

Type defenses

The effectiveness of each type on Blastoise.

NOR	FIR	WAT	ELE	GRA	ICE	FIG	POI	GRO
	½	½	2	2	½			

FLY	PSY	BUG	ROC	GHO	DRA	DAR	STE	FAI
							½	

Um Exemplo Fácil de Entender

Código em Python para representar o Blastoise

```
name =  "Blastoise"
ID = 9
HP = 79
AT = 83
DF = 100
SP_AT = 85
SP_DF = 105
SP = 78
moves = ["Bubble", "Tackle", "Tail Whip",
"Water Gun", "Bubble", "Water Gun", "Bite",
"Withdraw", "Skull Bash", "Hydro Pump"]
```

Um Exemplo Fácil de Entender

Código em Python para representar o Blastoise e o Charizard

```
name =  ["Blastoise", "Charizard"]
ID = [9, 6]
HP = [79, 78]
AT = [83, 84]
DF = [100, 78]
SP_AT = [85, 109]
SP_DF = [105, 85]
SP = [78, 100]
moves = [["Bubble","Tackle","Tail
Whip","Water Gun","Bubble","Water
Gun","Bite","Withdraw","Skull Bash","Hydro
Pump"], ["Ember", "Growl", "Leer",
"Scratch", "Rage", "Slash", "Flamethrower",
"Fire Spin"]]
```

Um Exemplo Fácil de Entender

Usando POO

- Criando uma classe:

```
class Pokemon:
    # Atributos
    name = ""
    ID = 0
```

- Criando um objeto ou uma instância:

```
pk1 = Pokemon()
pk1.name = "Blastoise"
pk1.ID = 9
```

- Acessando os atributos de um objeto:

```
print("O nome do Pokémon é {0} e seu ID é {1}".format(pk1.name,pk1.ID))
```

```
O nome do Pokémon é Blastoise e seu ID é 9
```

USANDO POO

- Até posso criar novos atributos, mas isso não é legal:

```
pk1.HP = 79
pk1.__dict__
```

```
{'ID': 9, 'name': 'Blastoise', 'HP': 79}
```

- Diferente de algumas outras linguagens, Não existem atributos ou métodos **privados (private)** em Python, tudo é **público (public)**, a convenção fala que se iniciarmos um atributo ou método com '_' ele é privado, porém, nada nos impede de alterá-los.

```
class Pokemon:
    _name = ""

pk1 = Pokemon()
pk1._name = "Blastoise"
```

USANDO POO

- Por isso, é melhor criar métodos para acessar e atribuir parâmetros. Adicionando métodos a uma classe:

```
class Pokemon:
    #métodos
    def atribuiNome(self, nome):
        self._name = nome
    def atribuiID(self, ID):
        self._ID = ID
    def info(self):
        print("O nome do Pokémon é {0} e seu ID é {1}".format(self._name,self._ID))
```

- Acessando os métodos de um objeto:

```
pk1 = Pokemon()
pk1.atribuiNome("Blastoise")
pk1.atribuiID(9)
pk1.info()
```

```
O nome do Pokémon é Blastoise e seu ID é 9
```

USANDO POO

- Mas pode dar um problema:

```
pk2 = Pokemon()
pk2.atribuiNome = "Charizard"
pk2.info()
```

```
Traceback (most recent call last):
  File "C:\Users\Anderson.Anderson-PC\Desktop\Python\Python -
Referências\Aulas\Aula 06\exemplos_aula_6.py", line 39, in <module>
    pk2.info()
  File "C:\Users\Anderson.Anderson-PC\Desktop\Python\Python -
Referências\Aulas\Aula 06\exemplos_aula_6.py", line 29, in info
    print("O nome do Pokémon é {0} e seu ID é
{1}".format(self._name,self._ID))
AttributeError: 'Pokemon' object has no attribute '_name'
```

USANDO POO

- Resolvemos isso adicionando o método **construtor** (__init__):

```
class Pokemon:
    #métodos
    def __init__(self, nome, ID):
        self._name = nome
        self._ID = ID
    def atribuiNome(self, nome):
        self._name = nome
    def atribuiID(self, ID):
        self._ID = ID
    def info(self):
        print("O nome do Pokémon é {0} e seu ID é {1}".format(self._name,self._ID))
```

- Criando um novo objeto:

```
pk2 = Pokemon("Charizard", 6)
pk2.info()
```

```
O nome do Pokémon é Charizard e seu ID é 6
```

USANDO POO

- Mas isso ainda funciona?

```
pk2.atribuiNome("Pikachu")
pk2.atribuiID(25)
pk2.info()
```

```
O nome do Pokémon é Pikachu e seu ID é 25
```

- E isso?

```
pk3 = Pokemon()
```

```
Traceback (most recent call last):
  File "C:\Users\Anderson.Anderson-PC\Desktop\Python\Python -
Referências\Aulas\Aula 06\exemplos_aula_6.py", line 42, in <module>
    pk3 = Pokemon()
TypeError: __init__() missing 2 required positional arguments: 'nome' and
'ID'
```

USANDO POO

- Mas dá pra corrigir:

```
class Pokemon:
    #métodos
    def __init__(self, nome = "", ID = None):
        self._name = nome
        self._ID = ID
    def atribuiNome(self, nome):
        self._name = nome
    def atribuiID(self, ID):
        self._ID = ID
    def info(self):
        print("O nome do Pokémon é {0} e seu ID é {1}".format(self._name,self._ID))

pk3 = Pokemon()
pk3.info()
```

```
O nome do Pokémon é  e seu ID é None
```

16

Modelando objetos com UML

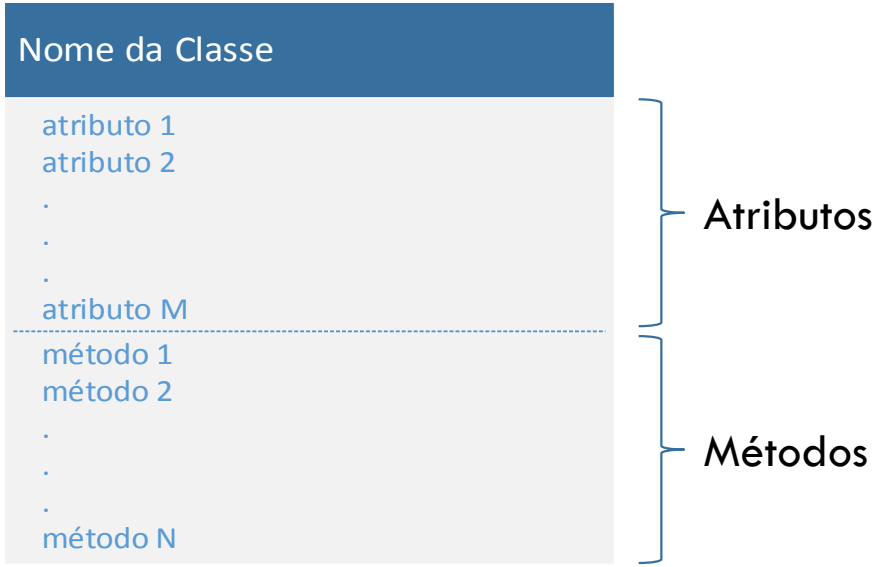
A forma padrão de modelar objetos

O QUE É UML

- ***Unified Modeling Language* (UML):** Notação padrão para a modelagem de objetos do mundo real.
  - Primeiro passo no desenvolvimento de um programa orientado a objetos
- Linguagem consistente para:
  - Especificar
  - Visualizar
  - Construir
  - Documentar

O QUE É UML

- Representação típica de uma classe:



Exercício para entrega

A Classe Pokémon completa

Classe Pokémon

- Criar uma classe Pokémon assim como a representada no diagrama UML abaixo, não se esqueça do método construtor:

Pokemon
nome: string ID: int HP: int AT: int DF: int SP_AT: int SP_DF: int SP: int golpes: string[]
atribuiHP(AT) atribuiAT(AT) atribuiDF(DF) atribuiSP_AT(SP_AT) atribuiSP_DF(SP_DF) atribuiSP(SP) aprendeGolpe(golpe)